

UNIDAD 1 · SEMANA 1

Introducción a los sistemas operativos

📅 Semana 1 · fechas por confirmar </> Laboratorio TD0 (resuelto + desafío IA)

~ 4 animaciones

MANUAL M1 · TEORÍA DE LA UNIDAD

Introducción a los sistemas operativos

Lectura base de la unidad 1 (≈ 1 h). Las cifras del equipo del curso salen de ejecutar los comandos del TD0 (Ubuntu 26.04, Linux 7.0).

CONTENIDO

§1	¿Qué es un sistema operativo?	§4	El arranque
§2	Capas y modos	§5	Dónde están hoy y cómo llegaron ahí
§3	Arquitecturas del núcleo		

§1 ¿Qué es un sistema operativo?

Un sistema operativo es el software que se interpone entre los programas y el hardware. Tiene dos misiones:

- **Administrar recursos.** Reparte el tiempo de CPU entre procesos, asigna y protege la memoria, ordena el acceso a discos, red y pantalla, y gestiona la energía. Resuelve los conflictos cuando varios programas quieren lo mismo a la vez.
- **Ofrecer una máquina más fácil de usar.** En lugar de sectores de disco, interrupciones y registros de controladores, los programas ven *abstracciones*: procesos, archivos, sockets, memoria virtual. Un programa que escribe en un archivo funciona igual con un SSD, un disco de red o una memoria USB.

Además, **protege**: un programa no puede leer la memoria de otro ni saltarse los permisos de un archivo. Esa protección depende de que el hardware tenga modos de ejecución distintos (§2).

§2 Capas y modos

De arriba hacia abajo: **aplicaciones** (bash, un navegador, tus programas) → **bibliotecas** (glibc, libssl) → **interfaz de llamadas al sistema** → **subsistemas del núcleo** (planificador, memoria, VFS, red) → **controladores** → **hardware**. Las dos primeras capas corren en **modo usuario**; las tres siguientes, en **modo núcleo**. La frontera entre ambas solo se cruza con una llamada al sistema (unidad 2).

No todo lo que hace una biblioteca necesita al núcleo: `printf` acumula texto en un búfer y `malloc` reparte memoria que ya pidió antes. Solo cuando hace falta el hardware —escribir de verdad, pedir más memoria— la biblioteca hace una llamada al sistema.

Practica esto → animación A1.1 · Capas del sistema

§3 Arquitecturas del núcleo

DISEÑO	QUÉ CORRE EN MODO NÚCLEO	A FAVOR	EN CONTRA	EJEMPLOS
Monolítico	Todo: planificador, memoria, sistemas de archivos, red, controladores	Rápido: una llamada, un cruce de ida y otro de vuelta	Un error en un controlador puede tumbar el sistema	Linux (con módulos cargables), los BSD
Micronúcleo	Lo mínimo: comunicación entre procesos (IPC), planificación, memoria	Aislamiento: un controlador que falla se reinicia	Más cruces y mensajes por operación	seL4 (verificado formalmente), QNX, MINIX 3
Híbrido	La mayoría de los servicios; algunos subsistemas en usuario	Compromiso entre los dos	Hereda parte de ambos problemas	Windows NT, macOS (XNU)

Linux es monolítico pero modular: en el equipo del curso hay **258 módulos** cargados (`lsmod`), controladores que se agregan y quitan sin reiniciar. Una cuarta idea, el *unikernel*, compila la aplicación y el núcleo en una sola imagen especializada, útil en la nube.

Practica esto → animación A1.2 · Arquitecturas del núcleo

§4 El arranque

1. **Firmware UEFI.** Prueba el hardware, inicializa memoria y buses y busca en su memoria no volátil (NVRAM) qué arrancar.
2. **Cargador de arranque** (GRUB o systemd-boot). Carga en memoria el núcleo (`vmlinuz`) y un sistema de archivos inicial (`initramfs`) con los controladores necesarios para montar el disco.

3. **Núcleo.** Se descomprime, configura la memoria virtual y los procesadores, detecta dispositivos, monta la raíz y lanza el **primer proceso, PID 1**.
4. **Espacio de usuario.** `systemd` (PID 1) arranca los servicios en paralelo hasta llegar al inicio de sesión. Todos los procesos descienden de él.

En el equipo del curso, `systemd-analyze` midió: firmware 9,1 s, cargador 4,9 s, **núcleo 2,6 s** y espacio de usuario 14,5 s (total 30,9 s). El núcleo es la etapa más corta.

Salida real

```
$ uname -srnm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod | tail -n +2 |
wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
237
28
Startup finished in 8.661s (firmware) + 3.568s (loader) + 3.253s (kernel) + 16.571s
(userspace) = 32.053s
graphical.target reached after 16.569s in userspace.
```

Practica esto → animación A1.3 · El arranque, y TD0 · 5

§5 ¿Dónde están hoy y cómo llegaron ahí

Hay sistemas operativos en los supercomputadores (desde noviembre de 2017 la lista TOP500 está dominada por completo por Linux; el n.º 1 de junio de 2026, LineShine, usa Kylin OS, una distribución basada en Linux), en los servidores de la nube, en los teléfonos (Android usa el núcleo Linux; iOS, el núcleo XNU), en los autos y equipos médicos (a menudo con micronúcleos de tiempo real como QNX) y en millones de dispositivos embebidos.

ÉPOCA	IDEA CLAVE
1950s	Procesamiento por lotes: un trabajo a la vez, sin interacción.
1960s	Multiprogramación y tiempo compartido (CTSS, Multics): varios usuarios a la vez, la CPU se reparte.
1969–1973	Unix en Bell Labs: procesos, archivos y tuberías como abstracciones simples; reescrito en C.
1980s– 1990s	Computadores personales; Linux (1991) y Windows NT (1993).
2007– 2010s	Teléfonos inteligentes, virtualización y contenedores en la nube.
2025– 2026	Sistemas operativos que alojan agentes de IA con identidad, permisos y auditoría propios (unidad 9).